Get unlimited access to the best of Medium for less than $1/week. Become a member

A Complete Database Normalization Tutorial

A Complete Guide on Functional dependencies, keys, Minimal Cover, BCNF, 1-NF, 2-NF, and 3-NF

Anuradha Wickramarachchi · Follow
Published in The Startup · 9 min read · Aug 26, 2020

199 1

I got the opportunity to be a tutor, for the last two years delivering Relational Databases. One important area of study in this particular subject was the database normalization. Students often find it challenging because they tend to miss out on important building block theories. Database normalization is a crucial step in structured data storage and data-warehousing for bigdata related applications.

In this article, I wish to gather all theories into one place making the understanding much easier. I hope this article helps you understand and apply functional dependencies in real applications.

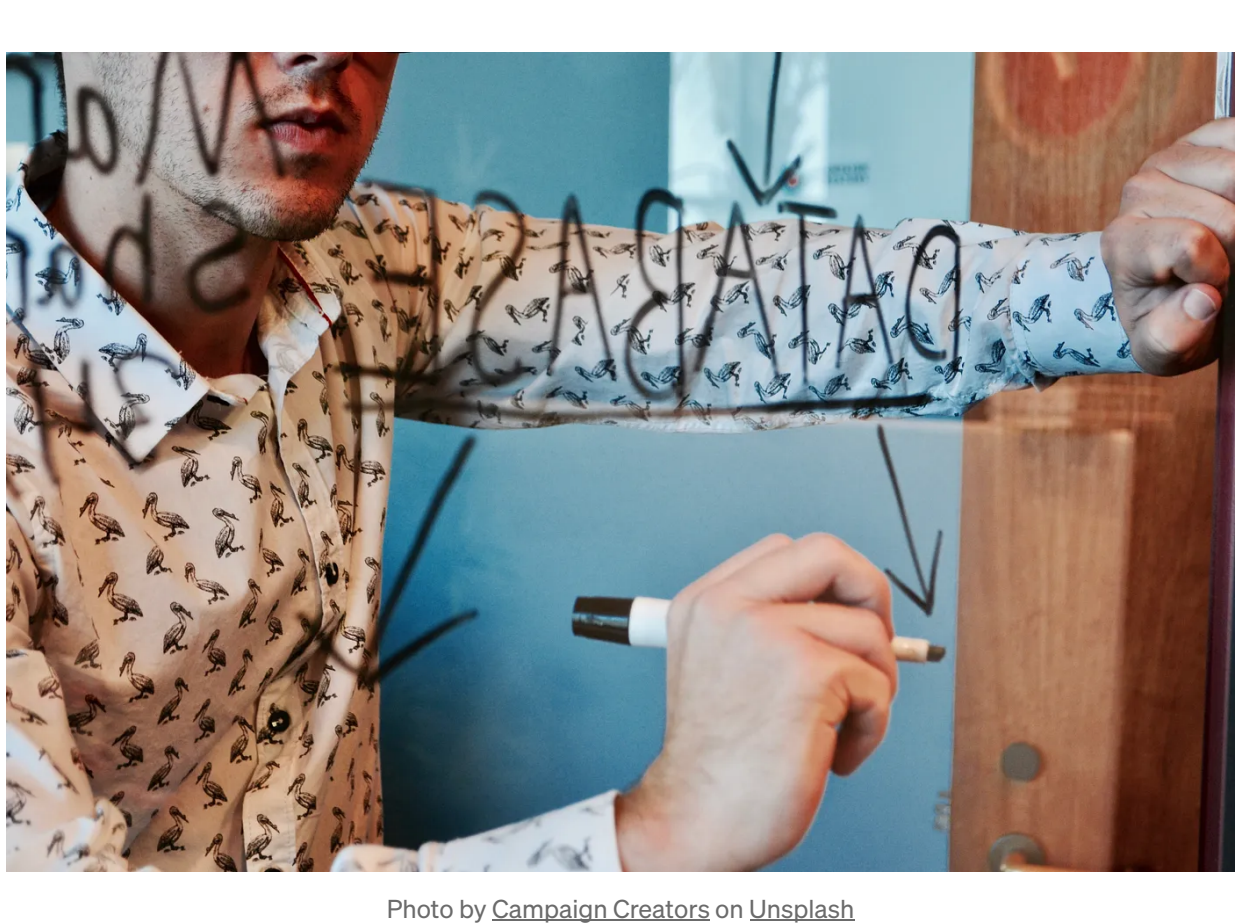


Photo by Campaign Creators on Unsplash

Functional Dependencies and Keys

A functional dependency is a constraint between two sets of attributes. In laymen's terms, they represent what attributes are related to what other attributes. For example, imagine a **STUDENT** database table for a university.

```
+--------+------------+-----------+-----+--------+
| id     | first_name | last_name | age | gender |
+--------+------------+-----------+-----+--------+
| 123456 | Peter      | Wick      |  38 |    m   |
+--------+------------+-----------+-----+--------+
| 123457 | Carl       | Renolds   |  48 |    m   |
+--------+------------+-----------+-----+--------+
| 123458 | Julia      | Victor    |  18 |    f   |
+--------+------------+-----------+-----+--------+
```

In this table, we can see the following relationships

- If you know `id`, we can determine all the other fields.
- Knowing `first_name` and `last_name` can retrieve other fields.

However, the chance of repeated names exists in a university. Hence, the only appropriate functional dependency is as follows.

```
id -> first_name, last_name, age, gender
```

We are certain that knowing gender or age will give us no clue as to what the other fields must be. Also, any combination of attributes including id can be a unique identifier for the university **STUDENT** table. Hence, we call them keys. However, the simplest key remains to be `id`, which we call as a candidate key. There could be several candidate keys, where one is picked to be the primary key. Let us consider the following functional dependencies.

```
R = (ABCDE), F = {A -> C, E -> D, B -> C}
```

Here, **R** represents the relation schema with attributes A, B, C, D, and E. **F** represents the functional dependencies among those attributes. Our first task is to find the candidate keys.

```
A->C ------------ (1)
E->D ------------ (2)
B->C ------------ (3)
-----
No singular attribute can determine anyone else, so we will try 2 combinations of attributes
(1) and (2)
(AE)+->ACDE
(BE)+->BCDE
(AB)+->ABC
-----
Still no luck, let us check the 3 combinations now. Note that we cannot have D in the left hand side as it is not contained in any of the determinants.
Combining (1) (2) and (3)

(ABE)+->ABECD
```

Note that I use the notation `(ABE)+`. This is called the closure, which means all the possible attributes that can be inferred from what we know in left. Pretty simple. Keep the following simple rules in mind as they will come in handy.

```
A->A trivial relationship
If A->B and B->C then A->C transitive
A->B them AX->B, we can add anything to left
```

Let us compute the closure for the attributes in the following schema and decide a primary key.

Note: attributes within a primary key are called prime attributes.

```
R = ABCDE, F = {A -> BE, C -> BE, B -> D}

A->BE   ----------------(1)
C -> BE ----------------(2)
B -> D  ----------------(3)

A->BE using (1) and (3) A->BDE (we add D to right)
Therefore; A+->ABDE

B+->BD
C+->CBED from (2) and (3)
D+->D (D does not exist as a determinant)
E+->E

None of the singular attributes can be a key, since none of them can determine everything else. So we can try combinations of 2.

Hint! when combining we need to combine with attributes that do not appear on the right. That way we expand the right hand side.

(AC)+->ABCDE --------- so AC is a KEY!!

We can see that no other combination of 2 attributes can be a key. However, look at this.

ABC is a key, but you can see that AC is within this and a key. So this does not qualify as a candidate key. So these kind of keys are called super keys. This cannot be a primary key.

The primary key is AC.
```

Key-finding Steps

- Obtain closures for individual attributes
- Consider combinations with 2 attributes (try the hint to reduce the dirty work)
- Expand and continue until a key is found

Now that we are aware of keys and how to find them let’s move on to the minimal cover algorithm. Many students find this quite challenging. However, if the steps are followed carefully the answer is guaranteed, without any gimmicks.

Minimal Cover

Minimal Cover is compressed of the fundamental set of functional dependencies that can represent the entire set of dependencies. Consider the following example. Here we will try to remove the redundant or inferrable functional dependencies.

```
R = ABCDE, F = {AE -> BDC, B -> C, C -> D, A->E}
```

How can we check if a functional dependency is redundant?

Step 1: Write the right-hand side as singular attributes

```
AE->B
AE->C
AE->D
B->C
C->D
A->E
```

Step 2: Simplify the left-hand side

```
Only AE can be further simplified. Can we get rid of A? For this to be possible E+ must contain A.
E+->E ------ A not found
Try A+
A+->AE (from the last dependency) So E is redundant. So we can replace AE with A

A->B --------(1)
A->C --------(2)
A->D --------(3)
B->C --------(4)
C->D --------(5)
A->E --------(6)

Note: This might be a bit more complex if there were more than 2 attributes in left. Still the same process, what to remove? check if rest of the attributes can infer the item to be remove. Simple!
```

Step 3: Remove redundancies

In order to remove redundancies, we follow a similar approach to that of step 2. If something to be removed, that dependency must be included among the rest of the dependencies.

```
Can we remove A->B? B should be within A+ just from other functional dependencies. Let's check.
A+->ACDE from (2),(3),(6) so we cannot remove A->B

Remove A->C?
A+->ABCDE from (1),(3),(4),(6) so we can remove A->C

Remove A->D?
A+->ABCDE from (1),(2),(4),(6) so we can remove A->D

remove B->C?
B+->B we cannot remove

Remove C->D
C+->C we cannot remove

Remove A->E, the only dependency inferring E
```

So the final answer or the closure is.

```
A->B
B->C
C->D
A->E
```

The order really matters in Minimal Cover computations. However, the answer will always be able to construct all the functional dependencies that existed.

Now that we have layered a solid understanding of functional dependencies, keys, and minimal cover we can proceed to study the normalization.

First Normal Form (1-NF)

This is the most naive normal form and most likely the data is already having this property satisfied.

- Table attributes must contain single values
- Each record must be unique

Consider the following table

```
+--------+--------+--------------------------+
| id     | name   | telephone_no             |
+--------+--------+--------------------------+
| 123456 | Robert | +1125697456, +4412345698 |
+--------+--------+--------------------------+
| 123457 | Julia  | +4513265474              |
+--------+--------+--------------------------+
```

We can see that the above table has multiple telephone numbers recorded for `Robert`. We can bring the table into the first normal form by splitting the entry for Robert into two rows.

```
+--------+--------+--------------+
| id     | name   | telephone_no  |
+--------+--------+--------------+
| 123456 | Robert | +1125697456   |
+--------+--------+--------------+
| 123456 | Robert | +4412345698   |
+--------+--------+--------------+
| 123457 | Julia  | +4513265474   |
+--------+--------+--------------+
```

Should the table contain duplicate rows, in 1-NF you will discard one of them to remove the redundancies.

Second Normal Form (2-NF)

The 2-NF normalization requires the following conditions to be satisfied

- Be in 1-NF
- Remove subsets of data belonging to multiple rows and place them in separate tables
- Use foreign keys to ensure the relationships are preserved

In our 1-NF table, you can see that records for `Robert` are duplicated. If he had more data, all of them will be duplicated consuming more space and inconsistencies. Hence, we can decompose the table into two more tables to attain 2-NF.

```
USERS TABLE
+--------+--------+
| id     | name   |
+--------+--------+
| 123456 | Robert |
+--------+--------+
| 123457 | Julia  |
+--------+--------+

USER PHONE NUMBERS TABLE

+--------+--------------+
| id     | telephone_no |
+--------+--------------+
| 123456 | +1125697456  |
+--------+--------------+
| 123456 | +1125697456  |
+--------+--------------+
| 123457 | +4513265474  |
+--------+--------------+
```

No in the `USERS` table we have user details and associated phone numbers in the table below. Note that `USER PHONE NUMBERS` table has `id` attribute which is a foreign key to the `USERS` table.

Photo by Marius Masalar on Unsplash

Third Normal Form (3-NF)

3-NF decomposition has several steps that involve the theories we discussed before. A relational schema R is said to be in 3-NF if its functional dependencies in the form `X -> A` satisfy one of the following conditions.

- X is a superkey
- A is a prime attribute

Let us walk through the steps while following an example.

```
R = ABCD, F = {A -> C, B -> C, AC -> D}
```

Step 1: Compute the Minimal Cover

```
A->C  ---------(1)
B->C  ---------(2)
AC->D ---------(3)

-->> Minimal Cover steps gives us the following dependencies
A->C
B->C
A->D
-->> Computing keys gives us AB to be a key

Proof that the above schema is not in 3NF
-----------------------------------------
A, B are prime attributes
But AC->D violates 3NF since, D is not a prime attribute and AC is not a key
```

Step 2: Group the functional dependencies by their left-hand side

```
A->CD
B->C
```

Step 3: For each grouped functional dependencies, gather them to form a Relational Shema

```
R1 = ACD
R2 = BC
```

Step 4: Remove redundant schemas

```
In this above example, we do not see any redundancies. So consider the following example.
Suppose we had R1=ABCD, R2=ACD and R3=DE

We can see the R2 is contained in R1, hence redundant. So get rid of R2. Fially we have R1=ABCD and R3=De.
```

Step 5: If the set of schemas do not have a superkey, add a super key table

```
We have R1=ACD where AC is a key. Hence R1 is a super key. Suppose we did not have such a schema, then we just add another schema R3=AC. Pretty simple. Without doing this we will not be able to join the decomposed tables.
```

Step 6: Project all the functional dependencies into the final set of schemas

```
We simply put the initial functional dependencies into the two tables. That's all
R1 = ACD, F1 = { A -> CD }
R2 = BC, F2 = { B -> C }
```

Photo by Scott Graham on Unsplash

Boyce-Codd Normal Form (BCNF)

A relational schema is said to be in BCNF if its functional dependencies are in the form `X -> A` such that `X` is a superkey. Note that this condition is more strict than 3-NF. Hence, there is a chance that some functional dependencies may not be visible in the final decomposition. This is something noteworthy!

Let's go through each step for BCNF with an example.

```
R = ABCD, F = { A -> BC, C -> D}
```

Step 1: Identify the non-trivial functional dependencies that violate BCNF.

```
A->BC
C->D

A is a key because A+->BCD
However, C->D is not because C+->CD
```

Step 2: Break the violating functional dependency `X -> Y` into two schemas as `R-Y` and `XY`.

```
Now R becomes R-D and CD;
R1 = R-D = ABCD - D = ABC
R2 = CD

Their functional dependencies are
F1 = { A -> BC }
F2 = { C -> D }

R1 and R2 do not violate BCNF since A and C are keys for R1 and R2.
If they did not, we'll have to break that again like this.
```

BCNF can also depend on the order of decomposition. Hence, multiple answers can exist!

Concluding Remarks

Database normalization is an easy task once the basic steps are understood. For BCNF one could easily use a tree and keep on decomposing. Although BCNF can kill certain functional dependencies, you must look for the closure of existing attributes to see if they are really lost or hidden. This is one major mistake I have seen among students.

I hope this was a useful article for readers with interest. Cheers!

Data Science
Database
Sql
Computer Science
Data

199 1

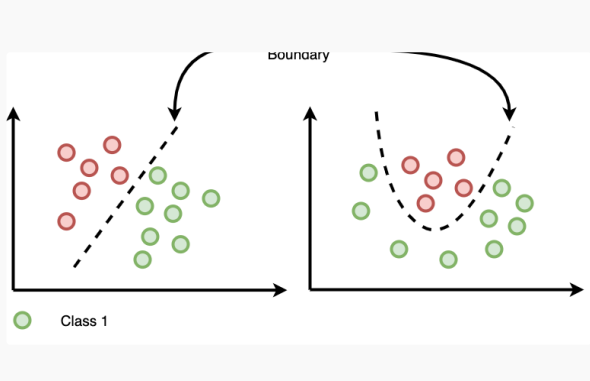
Written by Anuradha Wickramarachchi

2K Followers · Writer for The Startup

https://anuradhawick.com/

Follow

More from Anuradha Wickramarachchi and The Startup



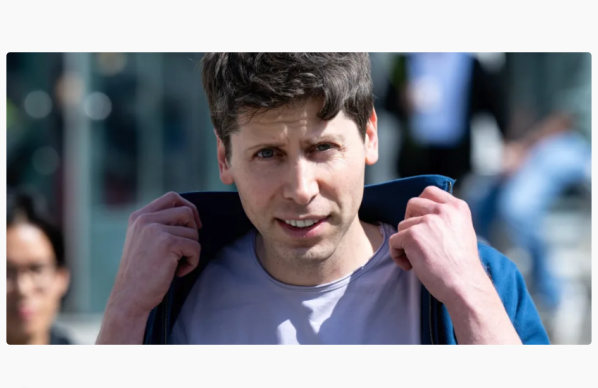
Anuradha Wickramarach... in Towards Data Scien...

Logistic Regression and Decision Boundary

Understanding logistic regression and its utility in classification

6 min read · May 19, 2020

170 2



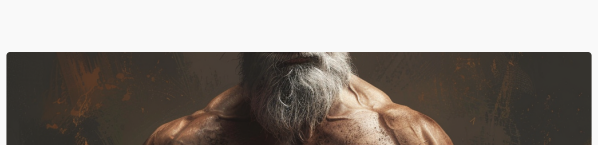
Michael Lim

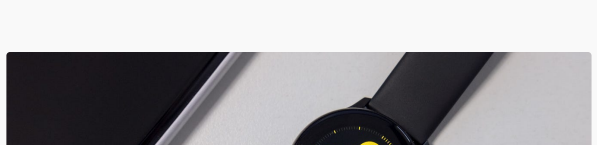
What Sam Altman’s Prediction About The $1B One-Person Busines...

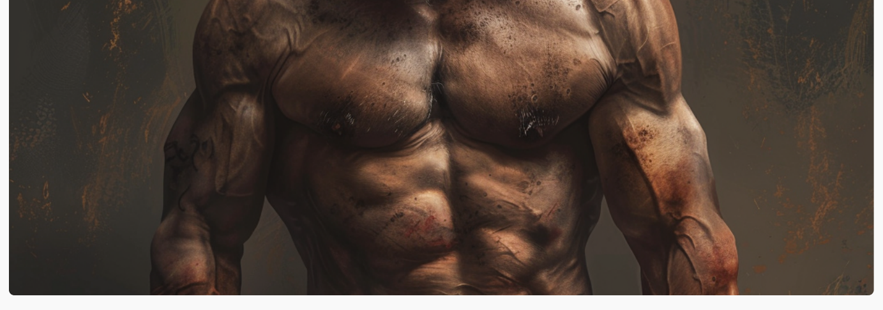
If you’re under 39, you’ve got a massive advantage.

· 5 min read · Feb 25, 2024

4.8K 128







Kurtis Pykes in The Startup

7 Habits That Are Incredibly Hard to Do But Pay Off Forever

Anything Worthwhile Requires a Great Deal of Suffering

· 6 min read · Feb 29, 2024

4.3K 76

Anuradha Wickramarach... in Towards Data Scien...

Extract Health Data From Your Samsung

Exporting and interpreting health data from Samsung Health

5 min read · Aug 22, 2020

122 1

See all from Anuradha Wickramarachchi

See all from The Startup

Recommended from Medium

Raghu Deshmukh

Indexing in SQL

What is Indexing?

3 min read · Oct 3, 2023

Rahul Sounder

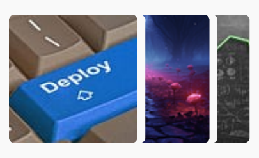
AWS Glue PySpark — Hands-on Coding for Data Engineers — ...

Below is the code to read data from the Athena AWS Glue Data Catalog Table

5 min read · Mar 24, 2024

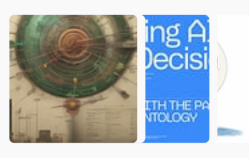
5

Lists



Predictive Modeling w/ Python

20 stories · 1057 saves

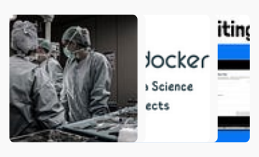


data science and AI

40 stories · 122 saves

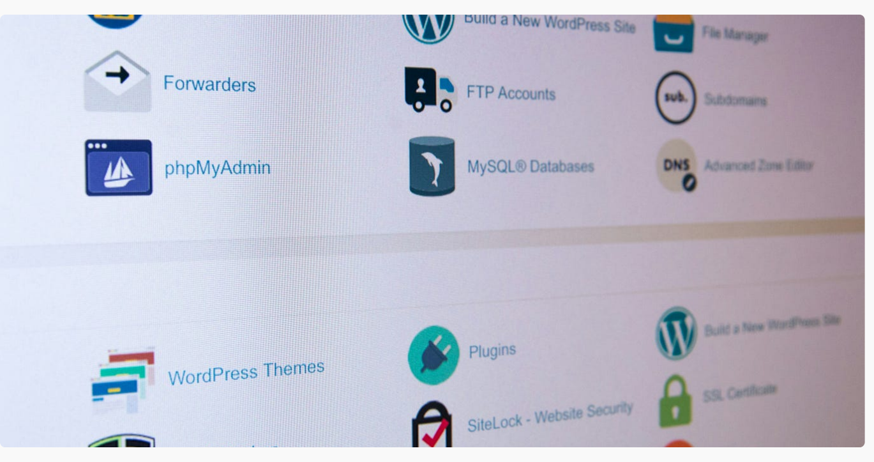
Practical Guides to Machine Learning

10 stories · 1264 saves



Coding & Development

11 stories · 537 saves



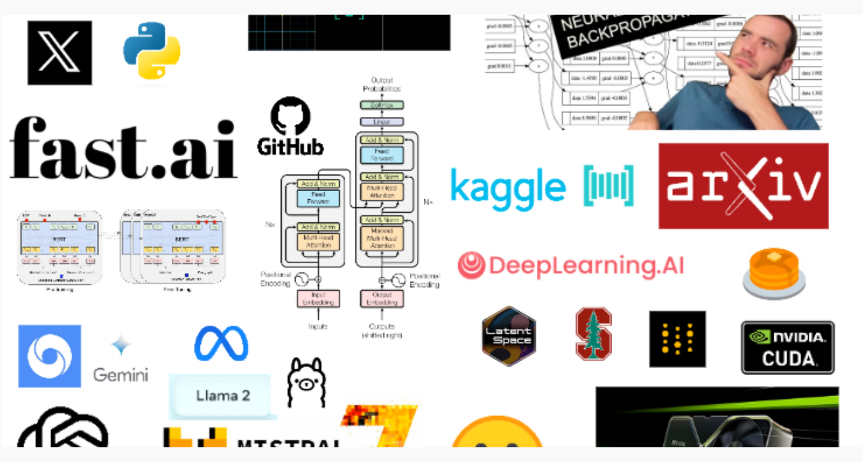
Suneel Kumar

20 Advanced SQL Techniques with Practical Examples

SQL (Structured Query Language) is a powerful language for managing and...

12 min read · Mar 14, 2024

30



Benedict Neo in bitgrit Data Science Publication

Roadmap to Learn AI in 2024

A free curriculum for hackers and programmers to learn AI

11 min read · Mar 10, 2024

9.1K 97

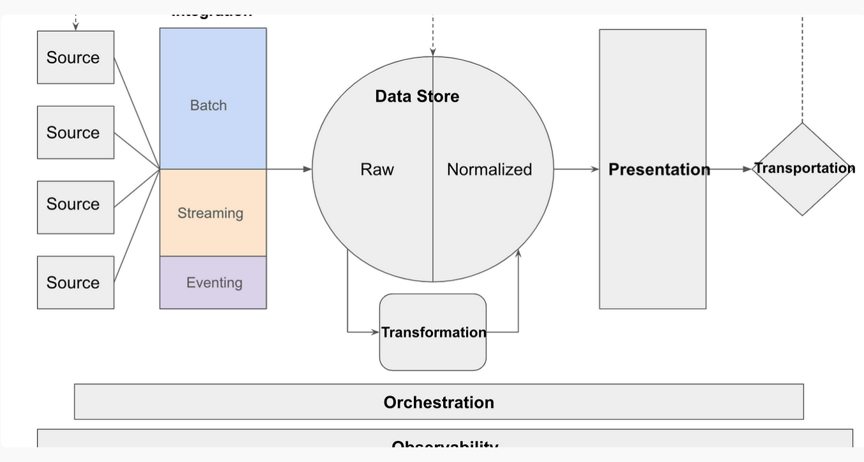
Shikha Chamoli

Quick SQL Hands-on — Window Functions

Welcome to the SQL hands-on series. Enhance your expertise in SQL with quick,...

7 min read · Mar 14, 2024

10



Dave Melillo in Towards Data Science

Building a Data Platform in 2024

How to build a modern, scalable data platform to power your analytics and data...

9 min read · Feb 5, 2024

2.6K 35

See more recommendations